

Objectifs :

- ✓ Comprendre la hiérarchie des exceptions en Java
- ✓ Créer et utiliser des exceptions personnalisées

Créez une classe CompteBancaire comme suit :

```
public class CompteBancaire {  
    private String numeroCompte;  
    private String titulaire;  
    private double solde;  
    private double decouvertAutorise;  
    private boolean bloque;  
    private Date dateCreation;  
    private ArrayList<String> historiqueOperations;  
  
    // Constructeur  
    // Getters et Setters  
}
```

1) Créer une classe BanqueException (exception parente)

2) Créez les classes suivantes qui héritent de BanqueException :

- a) SoldeInsuffisantException : Levée lorsqu'un retrait dépasse le solde disponible
- b) CompteInexistantException : Levée lorsqu'on tente d'accéder à un compte qui n'existe pas
- c) MontantInvalideException : Levée lorsqu'un montant est négatif ou nul
- d) DecouvertMaximumDepasseException : Levée lorsqu'un découvert autorisé est dépassé
- e) CompteBloqueException : Levée lorsqu'on tente d'utiliser un compte bloqué

3) Créez la Méthode déposer(double montant)

- Vérifie que le montant est positif (sinon lève MontantInvalideException)
- Vérifie que le compte n'est pas bloqué (sinon lève CompteBloqueException)
- Ajoute le montant au solde
- Ajoute l'opération à l'historique

4) Créez la méthode retirer(double montant)

- Vérifie que le montant est positif (sinon lève MontantInvalideException)
- Vérifie que le compte n'est pas bloqué (sinon lève CompteBloqueException)
- Vérifie que le nouveau solde (solde - montant) >= -decouvertAutorise
Si non, lève DecouvertMaximumDepasseException ou SoldeInsuffisantException
- Débite le compte
- Ajoute l'opération à l'historique

5) Créez la méthode transferer(CompteBancaire compteDestinataire, double montant)

- Utilise retirer() et déposer()

- Gère les exceptions (ne doit pas transférer si l'une des opérations échoue)

6) Créez la méthode `afficherHistorique()`

- Affiche toutes les opérations effectuées sur le compte

7) Créez la méthode `ajouterCompte (CompteBancaire compte)`

- Vérifie qu'un compte avec le même numéro n'existe pas déjà
- Lève `CompteInexistantException` si le compte est null
- Ajoute le compte à la collection

8) Créez la méthode `getCompte(String numeroCompte)`

- Recherche le compte par son numéro
- Lève `CompteInexistantException` si le compte n'existe pas
- Retourne le compte trouvé

9) Créez la méthode `effectuerVirement(String numeroSource, String numeroDestinataire, double montant)`

- Récupère les deux comptes (gère `CompteInexistantException`)
- Effectue le virement (gère les autres exceptions)
- Journalise les erreurs dans un fichier log

10) Créez la méthode `afficherTousLesComptes()`

- Affiche tous les comptes avec leurs informations

11) Dans la méthode principale :

- Ajouter 3 comptes valides
- Tenter d'ajouter un compte avec un numéro existant
- Déposer de l'argent
- Retirer de l'argent
- Effectuer un virement
- Tenter de retirer un montant supérieur au solde + découvert
- Tenter de déposer un montant négatif
- Tenter d'effectuer un virement vers un compte inexistant
- Tenter d'utiliser un compte bloqué